A04 下一次从哪里来，又在哪里停?

A04将A03的单次判断放进重复处理，并用容器保留记录；A05加入失败恢复，A08把内存数据写入文件。

循环决定下一轮从哪里来；容器和跨轮状态保存已得到的结果。结束时还要分清离开循环还是函数。

知识归类

实线表示按本章关注点归组，不表示先后执行，也不是唯一分类。

重复执行与多条数据的组织

重复与数据来源

按条件重复（while）

条件循环语句

每轮前重查条件，循环体负责推进所依赖的值；初次条件为假时一轮也不执行。

逐项遍历（for）

循环与数据来源

向数据来源依次取得下一项，耗尽后结束；取出一项不等于从原容器删除它。

生成序号范围（range）

提供整数序列供循环取用，range(n)从0到n之前；它提供值，不负责执行循环体。

循环中读取输入（input）

把取输入放进每轮才会取得新内容；只在循环外读一次无法收到后来的结束指令。

嵌套循环

外层每项再启动一轮内层遍历，适合行与座位等关系；内层结束后外层仍可继续。

容器与取值

有序记录列表（list）

容器类型

按位置保存多项，允许重复值；其中每一项称为元素，可逐项遍历，也可按索引取值。

按位置取值与计数（index / len）

索引从0开始，len取得数量而非最后位置；越界访问产生IndexError，不自动补项。

按键取值（dict）

容器类型与键值关系

按键保存对应的值，同键赋值替换旧值；直接读取缺失键产生KeyError。

判断是否包含（in）

判断某键是否在字典中，可先检查再取值；不等于在值中查找，也不同于for里的in。

列表中的字典

外层列表组织多条，内层字典组织字段；先按位置再按键取值，不自动保证字段齐全。

修改与跨轮状态

追加一项（append）

列表方法调用

把一项加入原列表末尾并返回None；与A02的strip返回新文字不同，不能靠接返回值取得列表。

布尔标记

把扫描中见过的事实跨轮保留，如曾有候补；循环前初始化，按需要更新而非每轮清空。

累计容器与初始化

跨轮状态安排

收集全部结果的容器在循环前建立，各轮添加；每轮重建只会留下当前轮的数据。

录入与汇总

录入把记录放进容器，汇总再遍历读取；保存到内存并不自动完成校验或持久化。

结束范围

自然结束

while因条件为假结束，for因没有下一项结束；离开后仍执行循环后方的代码。

离开循环（break）

立即离开最内层正在运行的循环，接着执行循环后的代码；不结束整层函数调用。

交回结果（return）

结束整个当前函数并交回结果；查到首条即返回时，不再走到函数末尾的未找到路线。

关键横向联系

同一知识点可以参与多种关系；箭头上的词才是连接含义。

循环中读取输入（input） —将正常项交给→ **追加一项（append）**

先检查结束指令再添加，避免把end也当成普通记录保存。

累计容器与初始化 —保留多条供读取→ **逐项遍历（for）**

录入结束后，另一段for可依次使用已收集内容，不必重新输入。

判断是否包含（in） —保护缺键读取→ **按键取值（dict）**

未知键不会自动忽略；只有判断后避开取值，才不会触发本次KeyError。

离开循环（break） —只退出内层→ **嵌套循环**

内层break不会同时终止外层；要交回整个函数结果则使用return。

追加一项（append） —原地增加一个元素→ **有序记录列表（list）**

元素是容器中的一项，方法是操作；names.append(name)改变列表，返回值却是None。

离开循环（break） —离开范围不同于→ **交回结果（return）**

break后仍可执行循环后的代码；return交回结果并结束当前函数。